



École d'ingénieurs de l'Université Sorbonne Paris Nord

SearchForAJob

Plateforme web de recherche d'emploi avec gestion de base de données relationnelle

**Rapport de SAE — Développement d'une application web dynamique
avec gestion de base de données**

Ablaye SOW — Yesser BENSELMA

Binôme

Encadrement pédagogique : Nathalie Pernelle, Manel Zarrouk, Samir Youcef
Année universitaire 2025 – 2026

1. Contexte et objectifs du projet

Dans le cadre de cette SAE, nous avons choisi de développer une plateforme de mise en relation entre candidats et recruteurs, sur le modèle d'un site d'offres d'emploi. Ce thème nous a semblé pertinent parce qu'il se prêtait naturellement aux trois types de relations attendues dans le sujet (1-1, 1-N, N-N), tout en restant assez riche pour justifier plusieurs entités distinctes : utilisateurs, entreprises, offres, compétences et candidatures.

L'application, que nous avons appelée SearchForAJob, permet à un candidat de consulter les offres disponibles, de filtrer sa recherche selon plusieurs critères (mot-clé, type de contrat, télétravail, niveau d'expérience, localisation, secteur), puis de postuler directement en ligne avec un CV et une lettre de motivation. Du côté recruteur, elle permet de créer une fiche entreprise, de publier une offre, de la modifier ou de la retirer, et de consulter les candidatures reçues.

2. Architecture générale et choix techniques

Conformément aux contraintes du sujet, l'application repose sur PHP 8 côté serveur, avec un accès à la base de données exclusivement via PDO : toutes les requêtes utilisent des paramètres liés, sans jamais concaténer une variable directement dans une chaîne SQL. Côté client, l'interface s'appuie sur Bootstrap 5 pour la mise en page et les composants visuels, complété par du JavaScript natif pour les interactions propres au projet : soumission automatique des filtres, ajout et suppression dynamique des champs de compétences lors de la publication d'une offre, ou encore apparition progressive des offres au défilement de la page.

Le code est organisé en plusieurs dossiers selon leur rôle. Le dossier public/ contient les pages accessibles depuis le navigateur, à raison d'un fichier PHP par page. Le dossier src/ regroupe les classes métier qui communiquent avec la base, une par entité (OffreModel, UserModel, EntrepriseModel, CandidatureModel...), ce qui évite de mélanger les requêtes SQL avec l'affichage. Le dossier config/ contient la connexion PDO et les fichiers de configuration sensibles. Le dossier assets/ regroupe les feuilles de style, les scripts JavaScript et les fichiers déposés par les utilisateurs, et sql/ contient le script de création complet de la base. Cette séparation nous a permis de travailler en parallèle sans trop nous gêner l'un l'autre.

3. Modélisation de la base de données

Le schéma relationnel s'appuie sur sept tables. La table utilisateur centralise l'authentification : email, mot de passe haché en bcrypt, rôle candidat ou recruteur. Chaque utilisateur est lié à exactement un profil, qui porte les informations personnelles (nom, téléphone, CV, biographie). Cette relation 1-1 n'est pas qu'une convention de notre part : elle est imposée par le schéma lui-même, grâce à une contrainte UNIQUE posée sur la colonne utilisateur_id de la table profil.

Un recruteur peut créer une entreprise, qui publie ensuite plusieurs offres : voilà la relation 1-N du modèle. Chaque offre peut à son tour requérir plusieurs compétences puisées dans un référentiel compétence, une même compétence (PHP, React, Docker...) étant naturellement partagée par plusieurs offres différentes. Cette relation N-N est résolue par une table de jointure, offre_competence, qui ne porte que les deux clés étrangères concernées.

La relation la plus riche du modèle reste celle entre candidats et offres : un candidat postule à plusieurs offres, une offre reçoit plusieurs candidatures. Nous l'avons modélisée par une table candidature qui, à la différence de offre_competence, porte des attributs propres : statut de la candidature, lettre de motivation, date. Ce n'est donc pas une simple table pivot mais une véritable entité-association entre candidat et offre.

Toutes les contraintes de clé étrangère sont posées avec ON DELETE CASCADE, afin que la suppression d'un utilisateur ou d'une offre entraîne proprement la suppression des données qui en

dépendent (candidatures, compétences associées...) sans laisser d'enregistrements orphelins en base. Le script complet, accompagné d'un jeu de données de test, se trouve dans `sql/schema.sql`.

4. Fonctionnalités développées

4.1 Front-end, base de données et logique métier (Ablaye Sow)

J'ai pris en charge la conception de la base de données ainsi que l'ensemble des pages PHP et de la logique métier associée. Cela couvre l'inscription et la connexion, avec vérification de l'adresse email par lien envoyé automatiquement et possibilité de réinitialiser son mot de passe en cas d'oubli ; la page d'accueil avec recherche par mot-clé et filtres combinables qui se soumettent automatiquement au changement, sans bouton de validation à chaque fois ; le tableau de bord recruteur, qui permet de créer sa fiche entreprise, de publier une offre avec une liste de compétences saisies librement (un champ par compétence, ajouté ou retiré au clic sans recharger la page), de la modifier ou de la supprimer, et de consulter les candidatures reçues avec le CV et la lettre de chaque candidat ; et le tableau de bord candidat, avec l'historique des candidatures et la modification du profil.

Plusieurs interactions JavaScript viennent compléter l'ensemble sans dépendre d'une librairie front-end supplémentaire : apparition progressive des offres au défilement grâce à l'API `IntersectionObserver`, compteur animé sur le nombre d'offres actives, et soumission automatique du formulaire de filtres dès qu'une case est cochée ou que le curseur de rayon est relâché.

4.2 Intégration d'une API externe (Yesser Benselma)

En complément des offres publiées directement sur la plateforme, le projet intègre une source externe via l'API Adzuna, qui agrège des offres d'emploi provenant de nombreux sites partenaires. Cette partie a demandé de comprendre la documentation de l'API (paramètres de recherche, format de réponse JSON, pagination), puis d'écrire un client HTTP dédié chargé de construire la requête, d'appeler l'API en HTTPS et de désérialiser la réponse reçue. Les résultats sont ensuite réaffichés sur une page propre au site, avec un rendu visuel cohérent avec le reste de l'interface, même si les données ne proviennent pas de notre base.

Un soin particulier a été apporté à la gestion des identifiants d'accès à l'API : ils ne sont jamais écrits en dur dans le code, mais chargés depuis un fichier de configuration local exclu du dépôt Git, ce qui évite de divulguer des clés sensibles en cas de publication du code sur une plateforme publique comme GitHub.

5. Difficultés rencontrées

Le développement n'a pas été linéaire. Une des premières difficultés concrètes a été de garder cohérents le nombre total d'offres affiché et la liste réellement filtrée : les deux passent par deux requêtes distinctes (l'une compte, l'autre récupère les lignes), et il fallait qu'elles appliquent exactement les mêmes critères, sous peine d'une pagination qui ne correspond plus à ce qui est affiché. Nous avons fini par factoriser la construction des conditions dans une seule méthode partagée par les deux requêtes.

Plus tard, la mise en place de l'envoi d'email (vérification de compte, mot de passe oublié) nous a fait découvrir une contrainte que nous ignorions complètement : Gmail refuse désormais toute connexion SMTP avec le mot de passe habituel du compte, et impose la génération d'un mot de passe d'application après activation de la validation en deux étapes. Une fois ce point compris, l'envoi en lui-même n'a posé aucune difficulté particulière.

6. Répartition du travail au sein du binôme

Membre	Contribution principale
--------	-------------------------

Ablaye Sow	Conception de la base de données, intégralité des pages PHP (authentification, recherche, gestion des offres, tableaux de bord) et intégration Bootstrap / JavaScript côté interface.
Yesser Benselma	Étude et intégration de l'API externe Adzuna, du client HTTP jusqu'à l'affichage des offres partenaires sur le site.

Le travail a été suivi par des commits réguliers sur le dépôt Git, chacun correspondant à une fonctionnalité ou une correction identifiable.

7. Limites actuelles et ouverture vers le NoSQL

Le modèle relationnel que nous avons mis en place fonctionne bien à l'échelle d'un projet académique, avec quelques centaines d'offres et d'utilisateurs de test. Il est intéressant de se demander ce qu'il se passerait dans un contexte de forte montée en charge, par exemple si SearchForAJob devenait une plateforme nationale avec plusieurs millions d'offres actives et des centaines de milliers de candidats ayant chacun postulé à plusieurs dizaines d'offres au fil du temps.

La page d'accueil, qui combine déjà aujourd'hui plusieurs filtres sur une jointure entre offre et entreprise, verrait son coût grimper fortement : chaque filtre supplémentaire est une condition combinée dans une même requête, et au-delà d'un certain volume de données, même de bons index ne suffisent plus à garantir un temps de réponse correct, parce que le moteur doit malgré tout parcourir et combiner plusieurs tables réparties sur le disque. La table candidature illustrerait bien cette limite : sur une plateforme à fort trafic, elle grandirait indéfiniment, et une requête en apparence simple comme « afficher toutes les candidatures reçues par un recruteur avec le profil du candidat et l'offre concernée » deviendrait une jointure à trois tables exécutée en continu. C'est précisément le genre d'opération où le relationnel montre ses limites de scalabilité horizontale : une base relationnelle classique se répartit beaucoup moins naturellement sur plusieurs serveurs qu'un système pensé dès l'origine pour être distribué.

Une alternative NoSQL orientée documents, comme MongoDB, permettrait de repenser le modèle en réduisant le nombre de jointures nécessaires aux lectures les plus fréquentes. Plutôt que de répartir les informations d'un candidat entre les tables utilisateur, profil et l'historique de ses candidatures, on pourrait stocker un document candidat unique embarquant directement la liste de ses candidatures comme sous-document, avec leur statut et leur date. La lecture d'un profil complet se ferait alors en une seule requête, sans jointure. De la même façon, une offre pourrait embarquer directement son tableau de compétences plutôt que de passer par une table pivot. Cette dénormalisation a un coût, notamment en cas de mise à jour d'une information dupliquée à plusieurs endroits, mais elle se prête nettement mieux à une distribution horizontale du stockage sur plusieurs serveurs, ce qui correspond bien aux besoins d'une plateforme devant absorber une charge de lecture très importante. Cette piste reste pour nous purement conceptuelle : nous n'avons pas cherché à l'implémenter, l'objectif de cette partie étant de réfléchir aux limites de l'approche relationnelle plutôt que de la remplacer dans le cadre de ce projet.

Conclusion

Ce projet nous a permis de mettre en pratique l'ensemble de la chaîne de développement d'une application web reposant sur une base de données relationnelle, depuis la modélisation jusqu'à l'interface utilisateur. Au-delà de l'aspect technique, travailler en binôme sur un dépôt Git partagé nous a obligés à organiser notre code pour qu'il reste lisible et modifiable par l'autre, ce qui n'est pas toujours le réflexe naturel quand on code seul. La réflexion finale sur les limites du relationnel nous a aussi permis de prendre du recul sur les choix faits au début du projet, et de mieux comprendre dans

quels cas une base de données comme la nôtre resterait pertinente, et dans quels cas elle ne le serait plus.

Livrables

- Lien du dépôt Git : [à compléter]
- Application fonctionnelle, testée sous environnement XAMPP
- Script SQL : sql/schema.sql
- README.md